

REQUISITOS FUNCIONAIS (RF)

RF01 — Cadastro de Usuário via API

O sistema deve permitir o cadastro de novos usuários exclusivamente por meio de uma requisição POST /register (via Thunder Client).

RF02 — Validação de Dados no Cadastro

O sistema deve validar se os campos “nome”, “email” e “senha” foram enviados antes de inserir o usuário no banco de dados.

RF03 — Armazenamento de Usuários no Banco MySQL

O sistema deve salvar as informações de cadastro na tabela usuários no banco area_login.

RF04 — Consulta de Usuário para Login

O sistema deve verificar o email e a senha enviados pelo usuário comparando os dados com o banco de dados.

RF05 — Login via Formulário HTML

O sistema deve receber os dados de login através do formulário presente em login.html, encaminhando-os para a rota POST /login.

RF06 — Resposta ao Login

O sistema deve enviar uma mensagem ao usuário informando se o login foi autorizado ou se os dados estão incorretos.

RF07 — Exibição da Tela de Login no Navegador

O sistema deve exibir apenas a página login.html como interface acessível pelo usuário final.

RF08 — Rota de Servir Arquivos Estáticos

O sistema deve disponibilizar arquivos HTML por meio da pasta public, permitindo acessar o login pelo navegador.

REQUISITOS NÃO FUNCIONAIS (RNF)

RNF01 — Linguagem de Desenvolvimento

O back-end deve ser desenvolvido em JavaScript utilizando Node.js.

RNF02 — Framework de Servidor

O sistema deve utilizar o framework Express para criar rotas e gerenciar requisições HTTP.

RNF03 — Banco de Dados Relacional

O sistema deve utilizar MySQL como banco de dados, acessado através do HeidiSQL ou equivalente.

RNF04 — Compatibilidade com Thunder Client

O sistema deve permitir que a rota de cadastro seja totalmente operável via Thunder Client ou ferramenta equivalente.

RNF05 — Estrutura de Pastas

O sistema deve ser organizado em arquivos separados, contendo:

- `server.js` (servidor)
- `db.js` (conexão ao banco)
- Pasta `public` para arquivos HTML

RNF06 — Porta do Servidor

O sistema deve operar na porta 3000 ou outra configurada pelo ambiente do Node.js.

RNF07 — Simplicidade da Interface

A interface deve ser minimalista, composta apenas por um formulário de login funcional.

RNF08 — Retorno em Texto/JSON

As respostas do servidor devem ser em formato texto (login) ou JSON (cadastro), sem interface adicional.

RNF09 — Performance Básica

O sistema deve responder às requisições HTTP em tempo razoável, mantendo funcionamento fluido em ambiente local.

RNF10 — Manutenibilidade

O código deve ser organizado, com rotas separadas e conexão modularizada no arquivo `db.js`.